TyKo

TyKo /taɪko/ is project aiming to achieve interoperability between Typst and JVM languages (primary Kotlin, but theoretically all of them).

Here's a roadmap, what it allows and what it plans to:

- [◧] Manipulating Typst values from JVM
  - [x] Basic types
  - [x] Content and other types
  - [x] Set, show rules
  - [◧] Functions (hiccups on closures that capture location, plugin or module)
  - [ ] Plugins, modules
- [◧] Calling Typst compiler from JVM
  - [x] Compilation (PNG, SVG, PDF, HTML)
  - [x] Query
  - [ ] Watching
  - [x] Evaluation
  - [x] World customization
    - [x] Library customization
      - [x] Standartized (`inputs`, `features`)
      - [x] Registering custom functions
    - [x] Fonts
    - [x] Styles
    - [x] Virtual file system
  - [ ] Realization (set/show rule expansion)
- [x] Calling JVM from Typst document
  - [x] Propagating JVM functions as Typst functions
  - [x] Loading JARs as plugins (Possible to achieve through other features, not supported directly)
  - [x] Freely throwing around JVM objects in Typst code
- [◧] Misc
  - [x] Downloading package from central repo
  - [x] Obtaining source code info (parse tree)
  - [ ] Annotation processor for writing overloads

Since version `0.4.0` the project is pure-JVM, self-contained, and doesn't require any platform-dependent binaries or preinstalled programs. However, using native binary might be a more fast options: see details in Drivers section

Contents

Contents 2
Quick[ish] Start 3
Model 10
    Basic types 11
        Primitives 11
        Float-like values 11
        Relative Length 11
        Collections 11
        Stroke 12
JVM Objects 12
Drivers 12

Quick[ish] Start

For the simplest code snippet, we will need two dependencies. In `gradle.kts` syntax:

```
implementation("org.ldemetrios:tyko-runtime:0.5.a")
implementation("org.ldemetrios:tyko-drivers-chicory-based:0.5.a")
```

Then, we need to create a TypstRuntime object. Let's for now not care what exactly it means.

```
val runtime = TypstRuntime { ChicoryTypstCore() }
```

Let's compile something to png. We will launch compilation in the virtual file system with a single file. By default, the entry point of the compilation is `"/main.typ"`

```
runtime.compilePng {
    files {
        singleFile("/main.typ", "Hello, world!")
    }
}
```

```
Warned<RResult<List<ByteArray>, List<SourceDiagnostic>>>:

Warned(output=Ok(value=[[B@38e79ae3]), warnings=[])
```

What did we get? Each compilation produces either success, or errors; and also potentially warnings. `Warned` is the type consisting of the result and warnings. `RResult` (Rust's `Result`, because there already is `Result` in Kotlin stdlib) is more or less "`Either`". `SourceDiagnostic` is an error or a warning. They contain all the useful stuff Typst's compiler tells us about what went wrong: the span in source file, the message, the trace. `List<ByteArray>` is the part that differs for compilation targets: it's `List<ByteArray>` for png, `List<String>` for svg, `ByteArray` for pdf, `String` for html.

This closely reflect how Typst's compiler thinks about compilation. Rust typically uses ADTs like `Result` or `Option` for represnting stuff with errors. However, Kotlin, following Java, uses exception. So there are a few methods that convert one approach to another:

Initial type	Method	Resulting type
`RResult<T, List<SourceDiagnostics>>`	`orThrow`	`T`
`Warned<RResult<T, List<SourceDiagnostics>>>`	`outputOrThrow`	`T`
`Warned<RResult<T, List<SourceDiagnostics>>>`	`warnedOrThrow`	`Warned<T>`

In case of the error, `TypstCompilerException` is thrown. TyKo tries to convert `Spans` from the original to `StackTraceElements`, appending them to stacktrace:

```
runtime.compilePng {
    files {
        singleFile("/main.typ", """
            #let f(x) = lorem(x)
            #let g(h) = h(-1)
            #g(f)
        """.trimIndent())
    }
}.outputOrThrow()
```

,

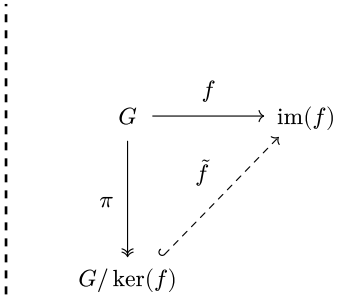
```
org.ldemetrios.tyko.compiler.TypstCompilerException: number must be at least zero
    at main.typ.f(main.typ:1)
    at main.typ.g(main.typ:2)
    at main.typ.<source>(main.typ:3)
    at org.ldemetrios.tyko.compiler.TypstCompilerException$Companion.construct(TypstCompilerException.kt:21)
    at org.ldemetrios.tyko.compiler.TypstCompilerException$Companion.construct$default(TypstCompilerException.kt:20)
    at org.ldemetrios.tyko.compiler.UtilsKt.orThrow(Utils.kt:12)
    at org.ldemetrios.tyko.compiler.UtilsKt.outputOrThrow(Utils.kt:15)
    at MainKt.main(Main.kt:31)
    at MainKt.main(Main.kt)
```

What else can we declare inside `files` block? File descriptor consists of, optionally, `PackageSpec`, and a virtual path `String`. By given descriptor, we return either the content of the file (byte array), or a `FileError`.

```
runtime.compileSvg {
    files {
        singleFile("/main.typ", """
            #set page(height :auto, margin : 20pt)
            #import "@preview/fletcher:0.5.6" as fletcher: diagram, node, edge
            #diagram(cell-size: 15mm, $
               G edge(f, ->) edge("d", pi, ->>) & im(f) \
               G slash ker(f) edge("ur", tilde(f), "hook-->")
            $)
        """.trimIndent())

        inPackage(null) { fd, session -> RResult.Err(FileError.NotFound(fd)) }

        fallback { fd, session -> runtime.resolvePackage(fd) }
    }
}.outputOrThrow().single().toByteArray()
```



Here, we have one file declared explicitly, then fallback for the rest of the files in package “null” (in the actual project, that is), and the fallback for all other files, which calls `resolvePackage`.

You can see, there still rises the problem of error handling mismatch. And if that is more convenient for your style, there is a hierarchy of `Exceptions`, mirroring the `FileError` adt, so you can instead write:

```
inPackageCatching(null) { fd, session ->
    throw FileErrorException.FileNotFoundException(fd)
}
```

Besides, you could’ve noticed there’s a `session` parameter in that lambda. It’s dedicated to handling JVM objects, but we’ll get to that later (see JVM Object chapter). For now, just ignore it.

What else do we have? Typst have another powerful mechanism, `query`. It returns not just a stuff of known format, but actual Typst values, which, in case of CLI, are serialized into json or yaml. Therefore, we need a model to represent Typst value. And we have that, a `TValue` hierarchy. Almost all of the types[1] have their respective `T<Name>` class.

```
val ratio = TRatio(TFloat(.7))
println(ratio.repr())
```

```
70.0%
```

For query to work, we need a selector.

```
val selector = THeading.ELEM.where("level" eq TInt(1), "outlined" eq TBool(true))
println(selector.repr())
```

```
heading.where(..arguments(..(), ..("level":1, "outlined":true, )))
```

Worth notice that `repr` of a `TValue` is not the same as `repr` inside Typst. For `TValues`, it only produces correct code, that evaluates to the value. For comparison, real `repr` for that selector would be `heading.where(level: 1, outlined: true)`.

```
runtime.query(selector) {
    files {
        singleFile("/main.typ", """
            = Introduction
            #lorem(10)
            #pagebreak()

            == Sub-Heading
            #lorem(8)

            = Discussion
            #lorem(18)
        """.trimIndent())
    }
}.outputOrThrow()
```

```
TArray<THeading>:

TArray(value=[THeading(body=TText(text=TStr(value=Introduction)),
level=TInt(value=1), depth=TInt(value=1), offset=TInt(value=0), numbering=TNone,
supplement=TText(text=TStr(value=Section)), outlined=TBool(value=true),
bookmarked=TAuto, hangingIndent=TAuto),
THeading(body=TText(text=TStr(value=Discussion)), level=TInt(value=1),
depth=TInt(value=1), offset=TInt(value=0), numbering=TNone,
supplement=TText(text=TStr(value=Section)), outlined=TBool(value=true),
bookmarked=TAuto, hangingIndent=TAuto)])
```

The good thing is, the type `TArray<THeading>` is auto-inferred from the selector: it's clear that, `heading.where(..)` select, well, headings. These types are propagated through selector construction (however, they might be lost during `selector.and` construction).

[1] At the time, `plugin`, `module` and `location` are not fully supported

The bad thing… Kotlin still doesn't support proper union types, and neither does Java. I racked my brains trying to fit all the required combinations into the hierarchy. To make matters worse, there are synthetic types, types with additional constraints. Like point is an array with exactly two elements of the same type. Et cetera, et cetera. Visit Model chapter if you want to know more.

There are two potential events, that might screw it completely and make me redesign model from the ground up. First, that might be Typst introducing custom types. In this case, TyKo would need to support those as well. Second, Kotlin team already has discussion about union-like types for errors. In current state it only could replace `RResult`, and couldn't be used for hierarchies containing generics (like `TArray` or `TDict` do), and I don't particularly like the idea of abandoning generics for those. Threads about Ceylon-like unions have been going around since at least 2012, but apparently in vain. But if at any future time they finally support union types, TyKo will migrate to use them.

Anyway. For now, TyKo doesn't support `watching`, although, it has long been requested, and it's one of the next big priorities. But TyKo has a function to `eval` content to a TValue. It's not like `eval` function in Typst: it tracks spans, it has access to VFS, in a word, it behaves like `compile`, but doesn't perform realization and layout:

```
runtime.eval {
    files {
        singleFile("/main.typ", """
            #metadata(1 + 2)
        """.trimIndent())
    }
}.outputOrThrow()
```

```
TValue:

TMetadata(value=TInt(value=3))
```

That's all with the operations, but not with customization. So far we've been only changing files. But there are few things we could set up.

Most notably, there is Library, what the Typst compiler considers to be its stdlib. This includes features (html and a11y), `inputs`, custom functions and definitions, and styles by default. For the library used by default, it's neither of the two features supported, empty inputs, and empty styles. You can also specify the specific FileDescriptor as entry point.

All right then.

```
runtime.compileHtml(FileDescriptor(null, "/start.typ")) {
    library {
        fresh(setOf(Feature.Html))

        inputs {
            "key"(1.t)
        }

        styles {
            add(TSetText(size = 20.0.t.pt))
        }

        values {
            "conifer"(TRgbColor(0x9f.t, 0xEB.t, 0x52.t, 0xFF.t))
        }
    }
    files {
        singleFile("/start.typ", """
            The input is: #sys.inputs.key

            #context text.size

            #conifer

            #text(fill: conifer)[This is not colored, because Typst doesn't support that]
        """.trimIndent())
    }
}.outputOrThrow()
```

```
String:

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
  </head>
  <body>
    <p>The input is: 1</p>
    <p><code data-lang="typc"><span style="color: #b60157">20pt</span></code></p>
    <p><code data-lang="typc"><span style="color: #4b69c6">rgb</span>(<span
style="color: #198810">"#9feb52"</span>)</code></p>
    <p>This is not colored, because Typst doesn’t support that</p>
  </body>
</html>
```

Functions are a little bit more complicated. In short, they accept `arguments` (represented by `TArgs<**>`), and optionally a context (represented by `TStyles`); and return either a result (`TValue`), or an error (String).

```
runtime.compileHtml {
    library {
        fresh(setOf(Feature.Html))

        func("println") {
            param("value") {
                input(TStr.TYPE)
                positional = true
                named = false
                required = true
            }

            returns(TNone.TYPE)

            pure {
                val value = it.positional[0] as TStr
                println(value.value)
                RResult.Ok(TNone)
            }
        }

        func("infer-size") {
            returns(TFloat.TYPE)

            contextual { _, ctx ->
                ctx.value.filterIsInstance<TSetText>()
                    .firstNotNullOfOrNull { it.size?.abs }
                    ?.let { RResult.Ok(it) }
                    ?: RResult.Err("No size is set")
            }
        }
    }
    files {
        singleFile("/main.typ", """
            #set text(size: 20pt)

            #println("Hello, world")

            #context infer-size()
            """.trimIndent())
    }
}.outputOrThrow()
```

```
Hello, world
```

```
String:

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
  </head>
  <body>
    <p>20</p>
  </body>
</html>
```

Well, yeah, RResult again. Even worse, it assumes the error is just a string, no stacktraces. But like that's gonna stop us! Of course, we have a catching option:

```
fun f(): Nothing = throw AssertionError("Error")
fun g(): Nothing = f()

runtime.compileHtml {
    library {
        fresh(setOf(Feature.Html))

        func("throw") {
            returns(TNone.TYPE)

            pureCatching {
                g()
            }
        }
    }
    files {
        singleFile("/main.typ", """
            #throw()
            """.trimIndent())
    }
}.outputOrThrow()
```

```
org.ldemetrios.tyko.compiler.TypstCompilerException: java.lang.AssertionError: Error
    at MainKt.main$lambda$1$f(Main.kt:23)
    at MainKt.main$lambda$1$g(Main.kt:24)
    at MainKt.main$lambda$1$0$0$0$0(Main.kt:34)
    at org.ldemetrios.tyko.runtime.NewFunctionSpecBuilder.pureCatching$lambda$0(LibrarySpecBuilder.kt:189)
    at main.typ.<source>(main.typ:1)
    at org.ldemetrios.tyko.compiler.TypstCompilerException$Companion.construct(TypstCompilerException.kt:21)
    at org.ldemetrios.tyko.compiler.TypstCompilerException$Companion.construct$default(TypstCompilerException.kt:20)
    at org.ldemetrios.tyko.compiler.UtilsKt.orThrow(Utils.kt:12)
    at org.ldemetrios.tyko.compiler.UtilsKt.outputOrThrow(Utils.kt:15)
    at MainKt.main(Main.kt:43)
    at MainKt.main(Main.kt)
```

Funny enough, the part before "main.typ" is actually a whole message, that just happen to contain formatted `StackTraceElements`.

Model

Model is a module that mostly consists of Kotlin classes representing Typst types. The supertype for them is `TValue`. Well, actually, there's a wider hierarchy, `IntoValue`. Well, actually, there's even wider type, `Argument`. Whyyyy?!

You see, Kotlin doesn't support union types. Typst, however, has union types all over. For example, `gradient.linear.stops` is an array. Array of what? Of either `colors` or `arrays`. But the latter aren't just arrays, they are arrays of size specifically 2, where the first element is `color`, and the second is `ratio|angle`. That sounds like a separate type to me, and it is: `TGradientStop`. Actually, it doesn't support angles. But that doesn't really matter. Types of such nature implement `IntoValue`, but not `TValue`. They have method `intoValue()` that returns the `TValue` that has the same Typst semantics.

But we were talking about union types. Sometimes, you can find a reasonable common name for stuff, like `TPaint` to describe either `TGradient`, `TColor` or `TTiling` (this name was adapted from original Typst compiler). But there are quite a lot of these unions. For that reason, an early version of TyKo had names like `TAutoOrColorOrFunctionOrGradientOrWhateverElseNeededToBeThere`. Which, with generics, made code really unreadable. And, even though there is now a much cleaner hierarchy, I would still advise you to not have those "unions" explicitly declared anywhere in your code.

What's an `Argument`, then? It's another union. There are places where you expect either `IntoValue`, or `Pair<String, IntoValue>`. Argument is exactly that — either "positional" `IntoValue`, or "named" `NamedArgument`, constructed with infix functions `eq` instead of `to`. It has already appeared in the manual:

```
THeading.ELEM.where("level" eq TInt(1), "outlined" eq TBool(true))
```

Also, it surely is cumbersome to create `TInt`, `TBool` etc by hand. At the time, there are extension `val t` properties:

```
1.t
```

```
TInt:

TInt(value=1)
```

They work for primitives, `Strings` and `ByteArrays`. In the future, it is planned to have annotation processor, that would generate necessary overloads, and extend TyKo with such overloads as well. Hypothetic example:

```
@TyKoOverloads(allKotlin = false)
operator fun TInt.plus(x: TInt) = TInt(this.value + x.value)
```

```
->
```

```
operator fun TInt.plus(x: TInt) = TInt(this.value + x.value)
operator fun TInt.plus(x: Long) = this.plus(TInt(x))
operator fun Long.plus(x: TInt) = TInt(this).plus(x)
```

With that out of the way, let's go through some types, which caveats do they have, etc.

Basic types

Primitives

For primitives, everything is simple. You have `TBool` for booleans, `TInt` for integrals (`i64`, `Long`), `TFloat` for floating point (`f64`, `Double`), `TStr` for strings.

Float-like values

These include `TAngle`, `TFraction`, `TRatio`.

```
val x = 1.0.t
println(x.deg)
println(x.rad)
println(x.pc)
println(x.fr)
```

```
TAngle(value=TFloat(value=0.017453292519943295))
TAngle(value=TFloat(value=1.0))
TRatio(value=TFloat(value=0.01))
TFraction(value=TFloat(value=1.0))
```

The common thing is, there are extension vals to mimic numeric literals. Kinda.

Relative Length

Typst has three different types for that, `ratio`, `length`, and `relative`, with the latter representing the sum of the former two. TyKo has... well... There are `TRatio`, `TLength`, and `TRelative`. The latter is supertype, except that for generic-compatibility reasons it's actually a typealias `TRelativeBase<*>`. The rest works the same, though:

```
println(1.0.t.pc + 2.0.t.em)
println(2.0.t.em)
```

```
TRelative(TFloat(value=0.0) pt, TFloat(value=2.0) em, TFloat(value=0.01))
TLength(abs=TFloat(value=0.0), em=TFloat(value=2.0))
```

Collections

There are `TArray` and `TDict`. They implement `List` and `Map`, respectively.

```
TArray(listOf(1.t, "2".t))
```

```
TArray<IntoValue>:

TArray(value=[TInt(value=1), TStr(value=2)])
```

And there is `TArgs`, which represents `arguments` type. It consists of positional and named arguments, and there's a helper function for mixing them:

```
TArgs.of(1.t, "key" eq 2.t)
```

```
TArgs<IntoValue>:

TArgs(positional=TArray(value=[TInt(value=1)]),
named=TDict(value={key=TInt(value=2)}))
```

They are generic and covariant, e.g. `TColor` <: `TPaint` ⇒ `TArray<TColor>` <: `TArray<TPaint>`.

Stroke

A lot things in Typst can be a stroke. Any paint is a stroke, length is a stroke. And there's a specific kind of dictionaries, which can be strokes as well. TyKo has a `TStroke` interface, which is implemented by all the aforementioned types, and for dicts there is synthetic type called, unoriginally, TStrokeImpl:

```
listOf(
    1.0.t.pt,
    java.awt.Color.RED.t,
    TTiling(TText(".".t))
)
```

```
List<IntoValue>:

[TLength(abs=TFloat(value=1.0), em=TFloat(value=0.0)),
TRgbColor(red=TInt(value=255), green=TInt(value=0), blue=TInt(value=0),
alpha=TInt(value=255)), TTiling(body=TText(text=TStr(value=.)), size=TAuto,
spacing=TPoint(x=TLength(abs=TFloat(value=0.0), em=TFloat(value=0.0)),
y=TLength(abs=TFloat(value=0.0), em=TFloat(value=0.0))), relativeTo=TAuto)]
```

– common type here is actually not denotable (contains intersections), but there's `TStroke` somewhere in there, trust me.

```
println((java.awt.Color.BLUE.t + 1.0.t.pt).repr())
```

```
stroke(cap:auto,dash:auto,join:auto,miter-limit:auto,paint:rgb(0,0,255,255,),thickness:(1.0pt),)
```

JVM Objects

Drivers

17.0.17